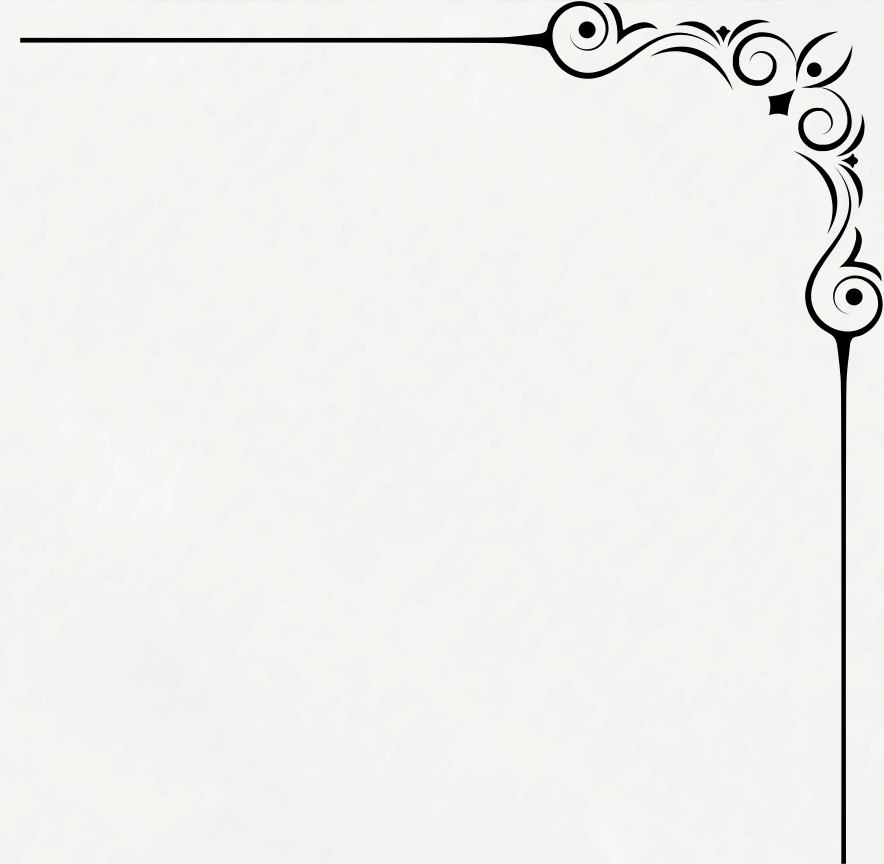
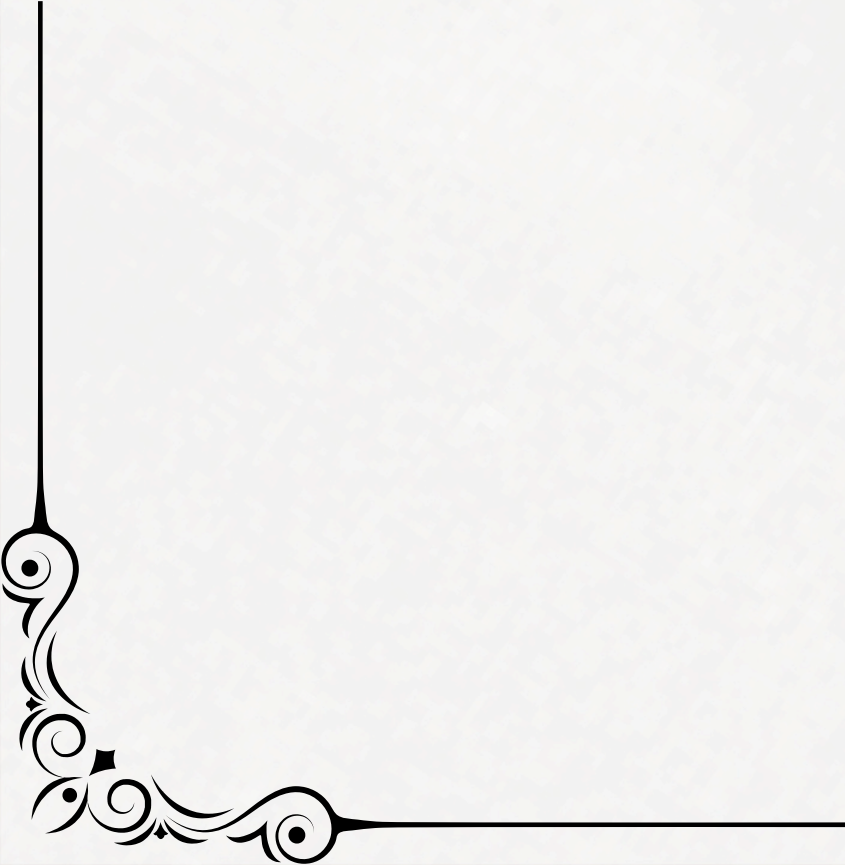




Cafe' Point.

PRINCIPLES OF SOFTWARE DESIGN

Of Sale System



ปัญหาและกลุ่มผู้ใช้

ปัญหาเดิม (Pain Points)

1. การรับออเดอร์หน้าร้านมีความยุ่งยากเมื่อต้องบันทึกรายละเอียดของเครื่องดื่ม เช่น
 - ระดับความหวาน
 - เมล็ดกาแฟ
 - รูปแบบการเสิร์ฟ
2. การแก้ไขราคา/โปรโมชั่นกระทบรายงานยอดขายย้อนหลัง
3. ขาดระบบจัดการท็อปปิ้งส่วนกลาง (Add-ons)

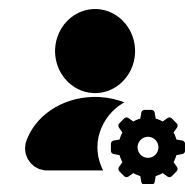
กลุ่มผู้ใช้งาน

CASHIER (พนักงานหน้าร้าน):



รับออเดอร์, ปรับแต่งเมนู, คัดเงิน (Cash/QR), ตัดสต็อก, พิมพ์ใบเสร็จ

ADMIN (ผู้ดูแลระบบ):



จัดการ User, เมนู, ท็อปปิ้งส่วนกลาง, โปรโมชั่น, ดูรายงานยอดขาย

ขอบเขตของระบบ

POS & Order Management:

- สั่งซื้อเครื่องดื่ม/ขนม
- ปรับแต่งตัวเลือก

Payment & Receipts:

- รองรับ Cash / QR Code
คำนวณเงินทอน
- ออกใบเสร็จ

Back-Office Administration:

- จัดการ User/Role
- จัดการ Products
Categories
- Global Add-ons

การออกแบบระบบ (Initial System Design)

Use Case Diagram: กำหนดขอบเขตการทำงานระหว่าง Cashier และ Admin

Activity & Sequence Diagram: ออกแบบ Flow การสั่งซื้อ การเลือกถือปิ้ง
การคำนวณส่วนลด และการชำระเงิน (Cash/QR)

Domain Model & Snapshot Strategy: ออกแบบการจัดเก็บราคาและส่วนลด ณ เวลา
ขายจริงเพื่อความถูกต้องทางบัญชี

เครื่องมือและเทคโนโลยีที่ใช้ (Tech Stack & Tools)

Frontend



Backend



Database

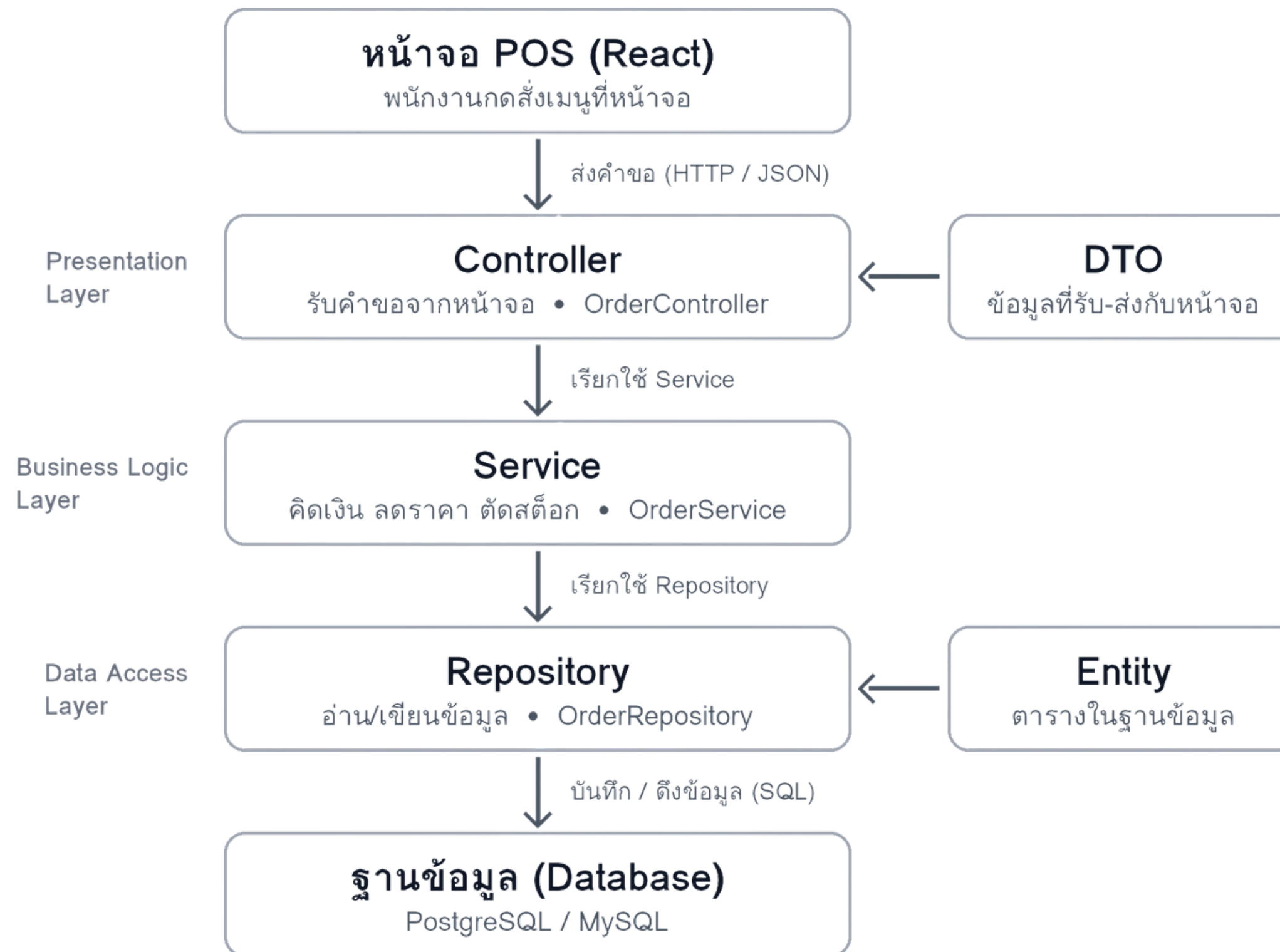


Testing & Tools



Deployment: Public Live URL [<https://poscafe.demo.com>]

Layered Architecture



ฟีเจอร์หลักของระบบ (Main Features)

- **POS & Order Management:** เลือกเมนู, ค้นหา, แยก Modal ปรับแต่งตัวเลือก เครื่องดื่ม (ความหวาน, คั่ว, ท็อปปิ้ง) และขนม
- **Payment & Receipts:** ชำระด้วยเงินสด (คำนวณเงินทอน) หรือ QR Code, บันทึกการชำระเงิน และออกใบเสร็จ
- **Back-Office Administration:** จัดการ User/Password, เมนูสินค้า, และระบบท็อปปิ้งส่วนกลาง (Global Add-ons)
- **Order History:** ค้นหาประวัติออเดอร์ย้อนหลัง และพิมพ์ใบเสร็จซ้ำ

Design Patterns

1. Strategy Pattern (service/discount/)

วัตถุประสงค์: สลับสูตรคำนวณส่วนลดโดยไม่แก้ไข OrderServiceImpl (ตาม OCP)

```
public interface DiscountStrategy {  
    DiscountType type();  
    BigDecimal calculate(BigDecimal subtotal, BigDecimal value);  
}
```

Design Patterns

2. State Pattern (domain/state/)

- **วัตถุประสงค์:** คุมการเปลี่ยนสถานะออเดอร์ (PENDING → PAID / CANCELLED) ป้องกันการแก้ไขเมื่อจ่ายเงินแล้ว

```
public interface OrderState {  
    void ensureModifiable();  
    OrderStatus cancel();  
    OrderStatus pay();  
}
```

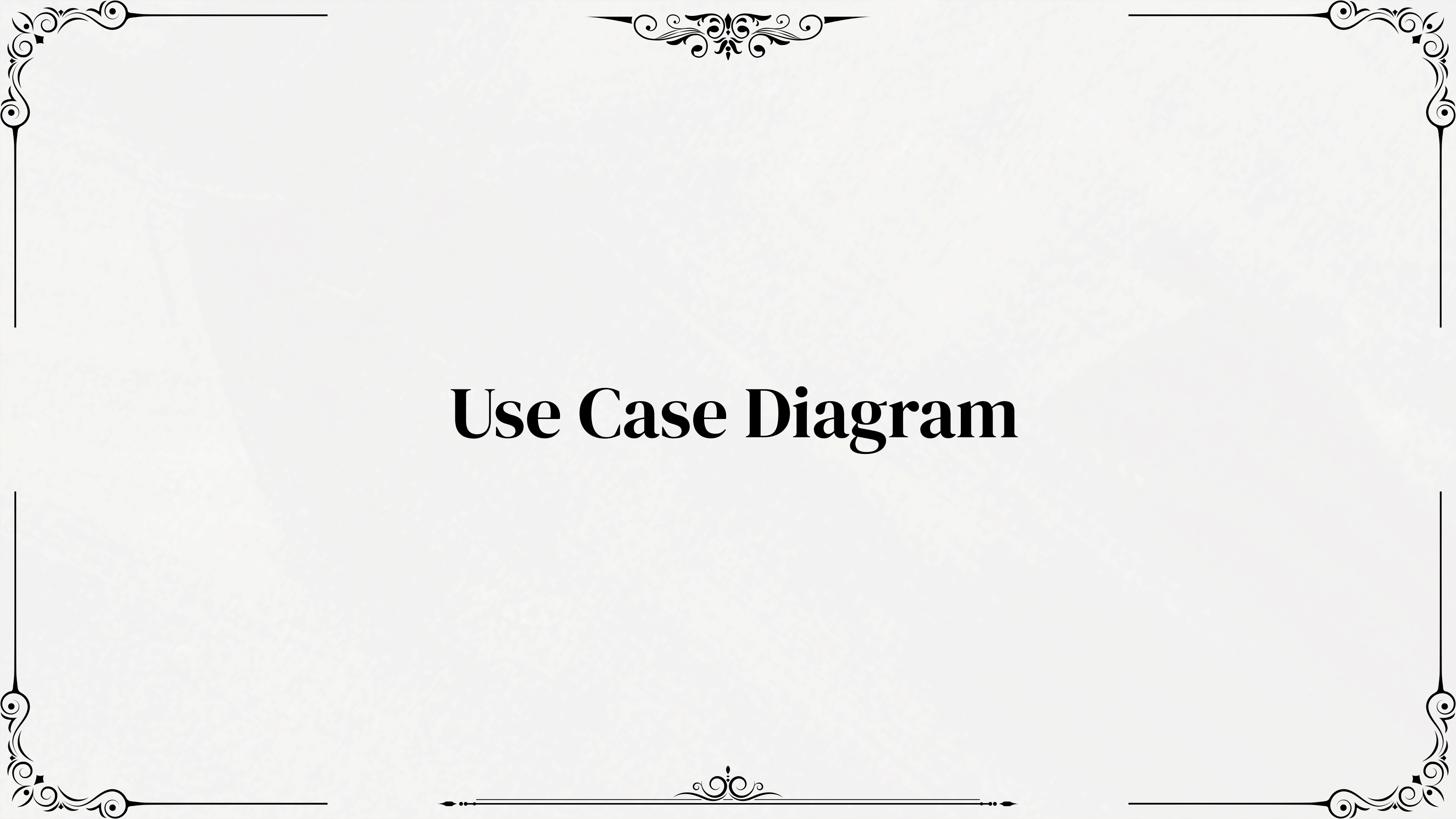
Design Patterns

3. Observer Pattern (service/listener/)

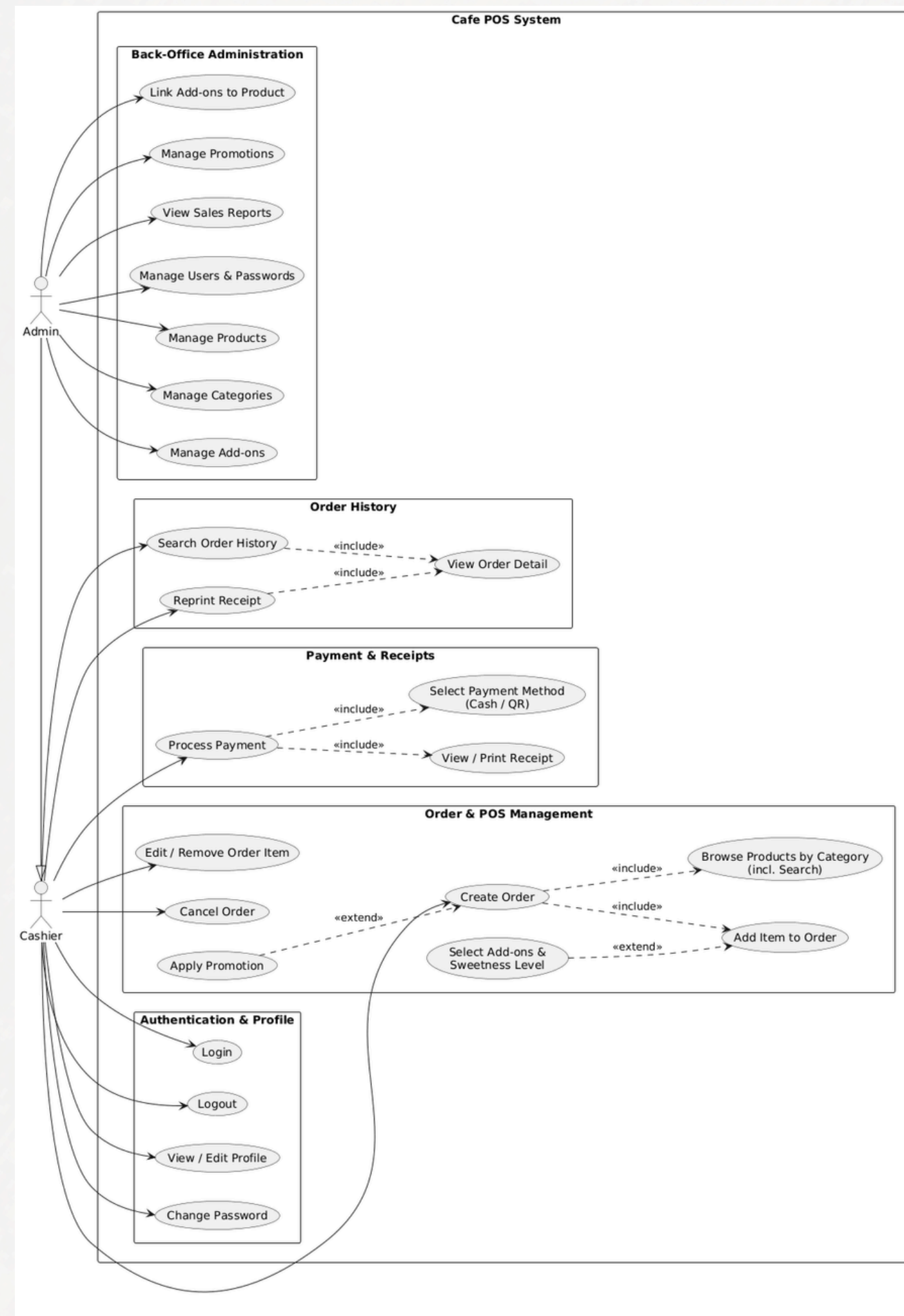
- **วัตถุประสงค์:** บันทึก Audit Log เมื่อชำระเงินสำเร็จผ่าน ApplicationEventPublisher
- **ประโยชน์:** แยก Logging Out ออกจาก PaymentServiceImpl ชัดเจน (Decoupling)

SOLID

- **S (SRP):** GlobalExceptionHandler จัดการ Error Response เท่านั้น / SaleAuditLogListener ทำหน้าที่บันทึก Audit Log แยกจาก Payment
- **O (OCP):** ใช้ DiscountStrategy Interface ในการคำนวณส่วนลด — เพิ่มส่วนลดรูปแบบใหม่ได้โดยไม่ต้องแก้ไข OrderServiceImpl
- **L (LSP):** OrderState ทุก Implementation (Pending, Paid, Cancelled) ทำงานตาม Contract เดียวกัน โดยไม่ throw UnsupportedOperationException
- **I (ISP):** แยก Service Interface ตาม Domain ชัดเจน เช่น OrderService, PaymentService, DiscountStrategy
- **D (DIP):** OrderServiceImpl ขึ้นตรงกับ Abstraction (Interface) ผ่าน Constructor Injection ทั้งหมด



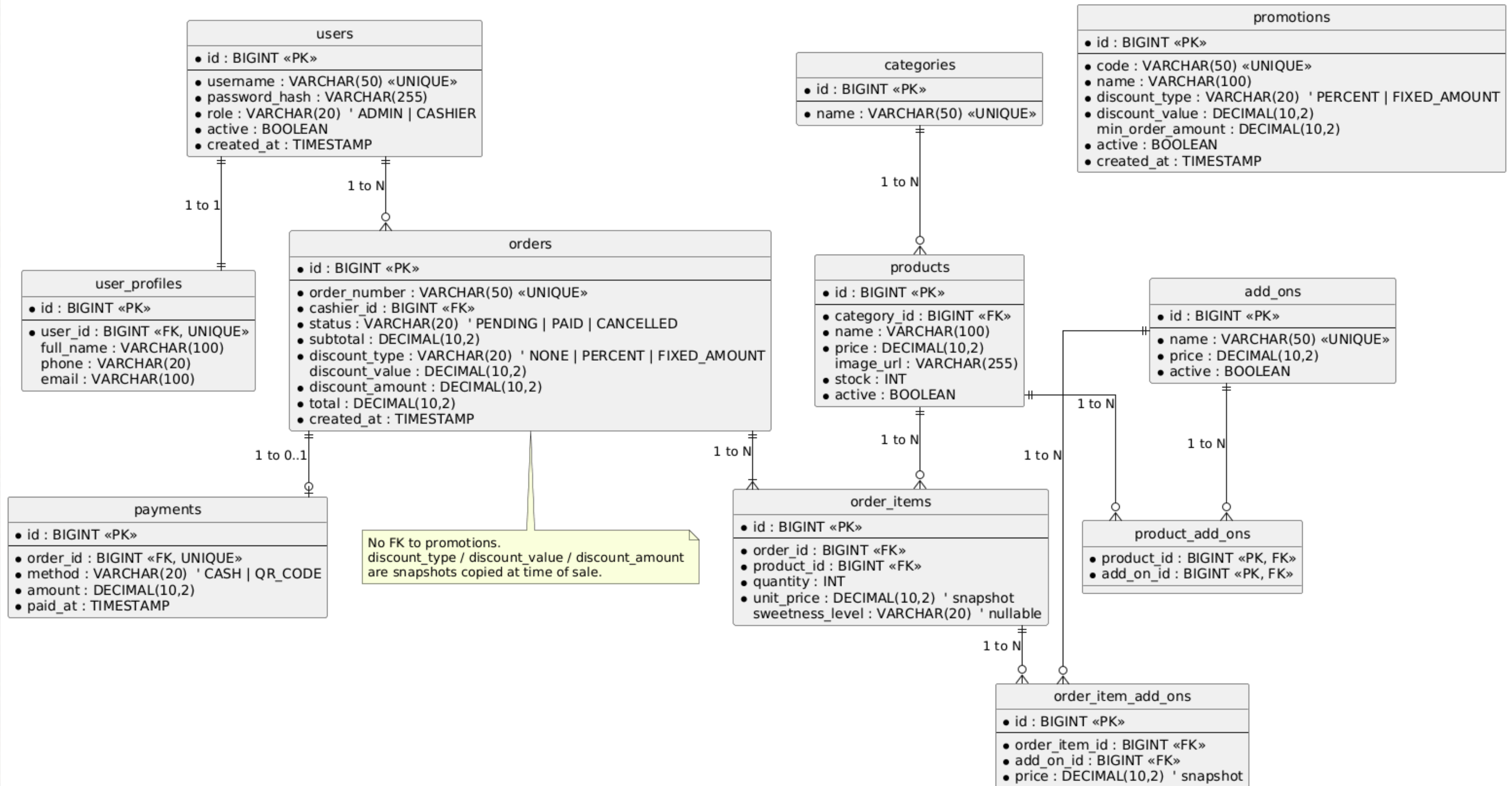
Use Case Diagram

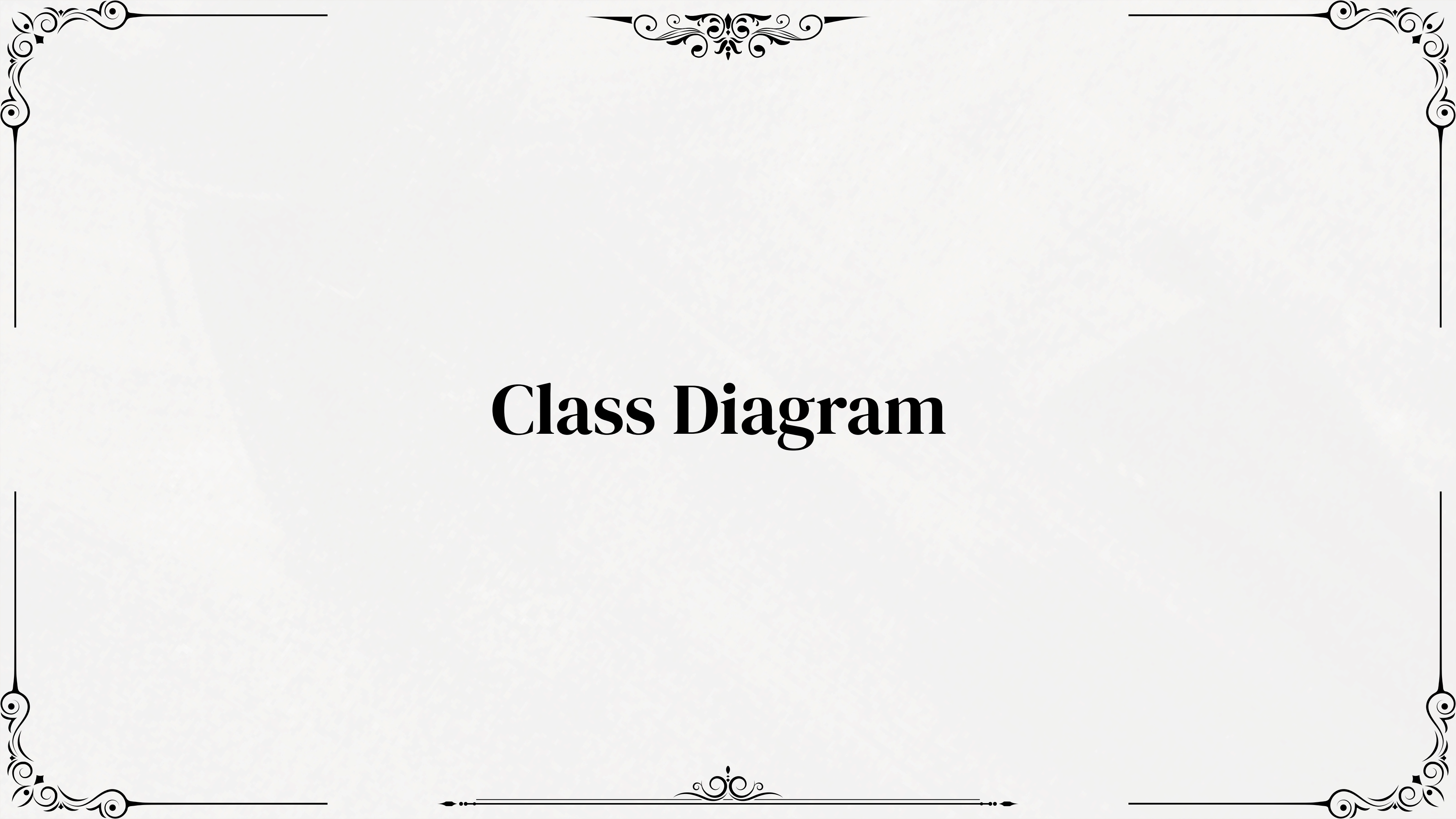




ER diagram

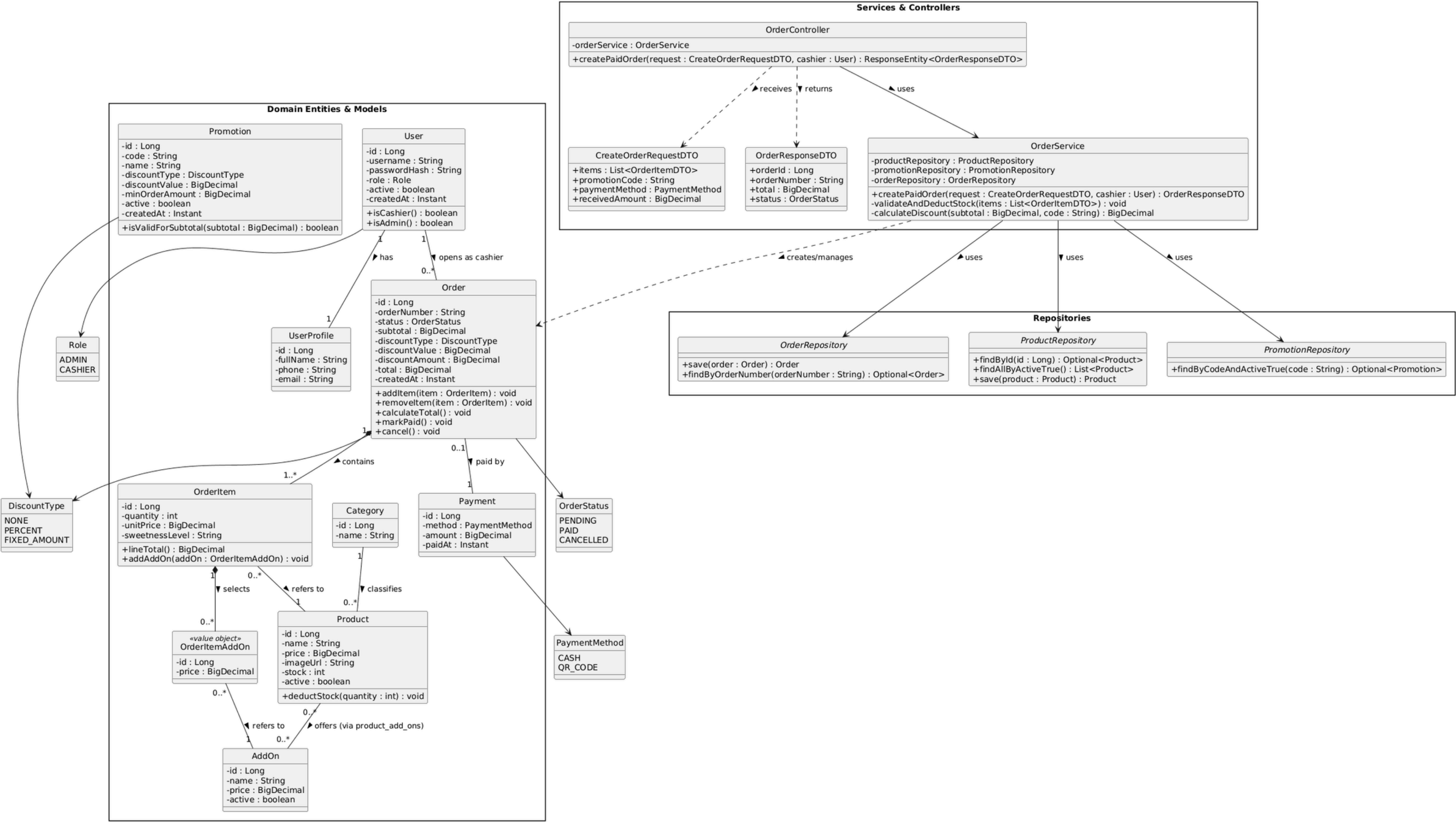
ER Diagram - Cafe POS System





Class Diagram

Class Diagram - Cafe POS System



บทเรียนที่ได้รับและแนวทางพัฒนาต่อ

ปัญหาที่พบ

- ความซับซ้อนในการจัดการ Database Relation
- Merge Conflicts ระหว่างการทำงานร่วมกัน

แนวทางปรับปรุง

- วางแผน ER diagram ให้ชัดเจน ก่อนเริ่มโค้ด
- ใช้ Git Workflow และทำ Code Review ให้เข้มงวดขึ้น



Q & A